



**SCHOOL OF INFORMATION TECHNOLOGY
RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA
BHOPAL (M.P.)**

CERTIFICATE

This is to certify that **Abhishek Mahto, Moksh Mahawar, Shabbir Husain, Shresth Raghuvanshi** of B.Tech fourth year, Computer Science and Engineering with specialization in Artificial Intelligence and Machine Learning have completed their Major Project-II entitled “**AI-Powered Video Knowledge Mapper**” during the period from January 2026 to June 2026 under the guidance and supervision.

We approve the project for the submission for partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering with specialization in Artificial Intelligence and Machine Learning.

Mr. Gajendra K. Ahirwar
Assistant Professor
Project Guide

Ms. Shrishti Raghuwanshi
Assistant Professor
Project Co-Guide

Dr. Jitendra Agrawal
Director
SOIT, RGPV



**SCHOOL OF INFORMATION TECHNOLOGY
RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA
BHOPAL (M.P.)**

APPROVAL CERTIFICATE

This is hereby certify that Minor Project-II entitled “**AI-Powered Video Knowledge Mapper**” being submitted by **Abhishek Mahto – 0002AL221005, Moksh Mahawar – 0002AL221034, Shabbir Husain – 0002AL221047, Shresth Raghuvanshi – 0108AI221062** of B.Tech fourth year, Computer Science and Engineering with specialization in Artificial Intelligence and Machine Learning to the School of Information Technology, RGPV, Bhopal.

Internal Examiner
Date:

External Examiner
Date:



**SCHOOL OF INFORMATION TECHNOLOGY
RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA
BHOPAL (M.P.)**

DECLARATION BY CANDIDATE

We, hereby declare that the work which is presented in the Major Project-II entitled “**AI-Powered Video Knowledge Mapper**” submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering with specialization in Artificial Intelligence and Machine Learning has been carried out at School of Information Technology, RGPV, Bhopal and is an authentic record of our work carried out under the guidance of **Mr. Gajendra K. Ahirwar** (Project guide) and **Ms. Shrishti Raghuvanshi** (Project co-guide) School of Information Technology, RGPV, Bhopal.

The matter in this project has not been submitted by us for the award of any other degree.

Abhishek Mahto [0002AL221005]

Moksh Mahawar [0002AL221034]

Shabbir Husain [0002AL221047]

Shresth Raghuvanshi [0108AI221062]



**SCHOOL OF INFORMATION TECHNOLOGY
RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA
BHOPAL (M.P.)**

ACKNOWLEDGEMENT

After the completion of Major Project-II, words are not enough to express our feelings about all those who helped us to reach our goal, feeling above all this is our indebtedness to the almighty for providing us this moment in life. First and foremost, we take this opportunity to express our deep regards and heartfelt gratitude to our project guide **Mr. Gajendra K. AHIRWAR**, and project co-guide **Ms. Shrishti Raghuvanshi** of the **School of Information Technology, RGPV, Bhopal**, for their inspiring guidance and timely suggestions in carrying out our project successfully. They have also been a constant source of inspiration for us.

We are extremely thankful to **Dr. Jitendra Agrawal, Director, School of Information Technology, RGPV, Bhopal**, for his cooperation and motivation during the project. We would also like to thank all the teachers of our department for providing invaluable support and motivation. We are also grateful to our parents and friends for their help and cooperation throughout this work.

Abhishek Mahto [0002AL221005]

Moksh Mahawar [0002AL221034]

Shabbir Husain [0002AL221047]

Shresth Raghuvanshi [0108AI221062]

TABLE OF CONTENTS

TITLE	PAGE NO.
Abstract	i
List of figures	ii
List of abbreviations	iii
Chapter 1 Introduction	1-3
1.1 Problem Statement	2
1.2 Purpose	2
1.3 Objectives	3
1.4 Scope	3
1.5 Constraints and Challenges	3
Chapter 2 Requirements Specification	4-6
2.1 Software Requirement	5
2.2 Hardware Requirements	6
2.3 Additional Components	6
Chapter 3 Problem Description	7-9
3.1 Limitations Of Traditional Video Learning	8
3.2 Challenges In Extracting Knowledge From Transcripts	8
3.3 Need For Automated AI-Driven Concept Mapping	8
3.4 Gaps In Existing Educational Tools	9
3.5 Educational And Skill-Development Needs	9
3.6 Real-World Relevance	9
Chapter 4 Feasibility Study	10-12
4.1 Technical Feasibility	11
4.2 Economic Feasibility	11
4.3 Operational Feasibility	11
4.4 Social Feasibility	12
4.5 Environmental Feasibility	12
4.6 Scalability	12
Chapter 5 Design And Implementation	13-19
5.1 System Architecture	14
5.2 Workflow Of The System	14
5.3 Data Flow Diagram & System Design	15
5.4 Generative AI Model Integration	16
5.5 Entity Relationship Diagram & Database Design	17

	5.6	Backend API Implementation	18
	5.7	Frontend & Dashboard Design	18
	5,8	Testing And Validation	19
	5.9	Final System Output	19
Chapter 6		Conclusion And Future Work	20-22
	6.1	Conclusion	21
	6.2	Future Work	22
Appendix-1		Glossary of terms	23-26
References			27



**SCHOOL OF INFORMATION TECHNOLOGY
RAJIV GANDHI PROUDYOGIKI VISHWAVIDYALAYA
BHOPAL (M.P.)**

ABSTRACT

The "**AI-Powered Video Knowledge Mapper**" (herein referred to as **TubeGraph**) is an advanced educational technology and data visualization system designed to dynamically extract, structure, and summarize core concepts from linear video content. As digital learning expands across platforms like **YouTube**, the sheer volume of unstructured video data makes rapid comprehension and targeted revision challenging. The resulting platform, **TubeGraph**, provides users with an intuitive dashboard that accelerates learning by transforming raw YouTube transcripts into semantic Knowledge Graphs.

The system's architecture integrates a high-performance, asynchronous backend built with **FastAPI** and a responsive, component-based frontend utilizing **React and React Flow**. By leveraging **Google's Gemini Flash Large Language Model (LLM)**, the engine performs real-time semantic analysis on extracted transcripts to identify key entities (*nodes*), map their relational dependencies (*edges*), and generate comprehensive **Markdown-formatted summaries**. To ensure high availability, environment reproducibility, and bypass aggressive cloud-bot mitigation protocols, the backend is containerized using **Docker** within a decoupled, hybrid-cloud architecture, piping data from a local extraction engine directly into a serverless **PostgreSQL database (NeonDB)**. Furthermore, user sessions and historical data are securely managed via stateless **JSON Web Token (JWT)** authentication and **SQLAlchemy ORM** models. Ultimately, this project demonstrates a robust, production-ready implementation of generative AI, providing users with an intuitive dashboard that accelerates learning and strengthens data comprehension.

LIST OF FIGURES

FIGURE NO.	TITLE OF FIGURE	PAGE NO.
5.1	<i>Flowchart illustrating user validation and data flow in the TubeGraph System</i>	15
5.2	<i>Entity-Relationship (ER) Diagram of the TubeGraph Database Storage</i>	17
5.3	<i>Screenshot of the Interactive TubeGraph Dashboard</i>	18

LIST OF ABBREVIATIONS

ACRONYM	FULL FORM
AI	Artificial Intelligence
API	Application Programming Interface
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
ORM	Object-Relational Mapping
UI	User Interface
UX	User Experience

Chapter 1

Introduction

CHAPTER-1

INTRODUCTION

The rapid growth of online video content has made it a major source of knowledge in education and training, but extracting meaningful insights from long videos remains challenging due to unstructured and lengthy transcripts. Learners often struggle to identify key concepts and relationships, making understanding less efficient. The AI Powered Video Knowledge Mapper addresses this issue by converting video transcripts into structured and interactive mind maps using generative AI. By integrating backend processing with intuitive frontend visualization, the system transforms raw textual data into clear conceptual representations, improving knowledge extraction, understanding, and overall learning efficiency.

1.1 PROBLEM STATEMENT

Video-based learning content is widely available, but extracting structured knowledge from it remains difficult. Most information is embedded in long transcripts, making it hard to identify key concepts and their relationships. Without proper visualization tools, users rely on manual summarization, which is inefficient and inconsistent. This project proposes an AI-based system that automatically converts video transcripts into structured and interactive mind maps to improve understanding and accessibility.

1.2 PURPOSE

- To design and implement an AI-based system that converts video transcripts into structured mind maps.
- Enable automatic extraction of key concepts and relationships from unstructured video content.
- Simplify understanding through interactive visualization of knowledge using node-edge representations.
- Provide a scalable and extendable platform for future AI-based learning and visualization systems.

1.3 OBJECTIVES

- Build a scalable backend using FastAPI to process transcripts and integrate generative AI models.
- Develop structured JSON outputs representing nodes and relationships for mind map generation.
- Create interactive visualization components using React.js, React Flow, and Dagre.
- Maintain clarity in documentation and system design to support evaluation and future development.

1.4 SCOPE

- **Educational Use:** Helps students understand video content through structured and interactive mind maps.
- **Practical Application:** Supports knowledge extraction from lectures, tutorials, and online courses.
- **Visualization:** Provides clear representation of concepts and their relationships.
- **Extendibility:** Can be expanded with features such as multi-video comparison and advanced AI capabilities.
- **Generalization:** Applicable across different domains including education, training, and content analysis.

1.5 CONSTRAINTS AND CHALLENGES

- **Handling Large Transcripts Efficiently:** Ensuring smooth processing of long video content without performance issues.
- **AI Output Consistency:** Maintaining accurate and structured mind map generation from generative AI models.
- **Visualization Complexity:** Ensuring clarity and readability in dynamically generated node-edge diagrams.
- **API Dependency:** Managing latency and reliability of external transcript and AI services.
- **Scalability:** Designing the system to handle multiple users and large volumes of data effectively.

Chapter 2

Requirements

Specification

CHAPTER-2

REQUIREMENTS SPECIFICATION

2.1 SOFTWARE REQUIREMENTS

1. OPERATING SYSTEM

- Windows 10/11 (Recommended) or Linux (Ubuntu 20.04 or newer).
- Compatible with Python environments, frontend frameworks, and API integrations.

2. PROGRAMMING LANGUAGES

- **Python:** Core language for backend development, API integration, and data processing.
- **JavaScript:** Used for frontend development and interactive visualization.

3. LIBRARIES AND FRAMEWORKS

- **FastAPI:** Backend framework for handling API requests and system integration.
- **React.js:** Frontend framework for building the user interface.
- **React Flow:** For rendering interactive nodes and edges of the mind map.
- **Dagre:** For automatic layout and positioning of nodes and edges.
- **Requests / HTTP libraries:** For API communication and data exchange.
- **YouTube Transcript API:** For extracting video captions and transcripts.
- **Generative AI API (Gemini Flash):** For processing transcripts and generating structured mind maps.

4. DEVELOPMENT AND TESTING TOOLS

- **Jupyter Notebook / VS Code / Google Colab:** For coding, testing, and experimentation.
- **Git & GitHub:** Version control and collaborative development

2.2 HARDWARE REQUIREMENTS

1. COMPUTING DEVICE

- Standard academic machine (Windows/Linux) with at least 8GB RAM.

2. PROCESSOR

- Intel i5 or equivalent (No GPU required due to API-based AI processing).

3. STORAGE

- Minimum 256GB for storing project files, transcripts, and generated outputs.

4. NETWORK CONNECTIVITY

- Required for accessing YouTube transcripts, AI APIs, and frontend-backend communication.

2.3 ADDITIONAL COMPONENTS (for expansion of this project)

- Cloud Deployment Tools (e.g., AWS, Azure, or Docker) for scalability and online access.
- Database Systems (e.g., MongoDB, PostgreSQL) for storing transcripts and generated mind maps.
- Advanced Visualization Tools for enhanced graphical representation and user interaction.

Chapter 3

Problem Description

CHAPTER-3

PROBLEM DESCRIPTION

3.1 LIMITATIONS OF TRADITIONAL VIDEO LEARNING

- Manual note-taking from long-form videos is time-consuming and often misses key structural connections.
- Learners struggle to visualize the hierarchy of complex topics presented linearly in a video format.
- Rewatching entire videos to find specific conceptual relationships is inefficient and non-scalable.
- Passive consumption of video content often leads to lower retention of interconnected ideas.

3.2 CHALLENGES IN EXTRACTING KNOWLEDGE FROM TRANSCRIPTS

- Video transcripts are often unstructured, long-form blocks of text that are difficult to scan.
- Identifying core entities and their logical relationships within a raw transcript requires high cognitive load.
- Automated transcripts frequently contain 'noise', i.e., filler words, repetitions, or grammatical errors, that obscure meaning.

3.3 NEED FOR AUTOMATED AI-DRIVEN CONCEPT MAPPING

- Generative AI can parse thousands of words in seconds to identify recurring themes and logical nodes.
- Automated mapping reduces the time between "consuming" information and "organizing" knowledge.
- AI models can provide varying levels of detail (via temperature settings) to suit different learning speeds.

3.4 GAPS IN EXISTING EDUCATIONAL TOOLS

- Most video platforms lack built-in tools for interactive, non-linear knowledge visualization.
- Existing mind-mapping software requires manual entry of every node, offering no direct integration with video content.
- A lack of unified dashboards that combine video playback, transcript extraction, and real-time flowcharts.
- Current AI summarizers provide text paragraphs rather than interactive, spatial representations of data.

3.5 EDUCATIONAL AND SKILL-DEVELOPMENT NEEDS

- Students need hands-on experience integrating Large Language Models (LLMs) like Gemini into full-stack applications.
- Developing this project builds practical skills in React Flow, Python (FastAPI), and complex JSON data handling.
- Learning to manage asynchronous tasks (fetching transcripts while generating AI nodes) prepares students for real-world software engineering.
- The project encourages mastering UI/UX challenges, such as auto-arranging nodes using Dagre for optimal readability.

3.6 REAL-WORLD RELEVANCE

- Applicable in EdTech for creating automated study guides and curriculum maps for online courses.
- Useful for corporate training and technical documentation where complex workflows are explained via video.
- Supports rapid knowledge transfer in research and development sectors by mapping out webinar and seminar content.
- Forms a foundation for advanced ‘Knowledge Graphs’ used in personalized AI tutoring and enterprise data management.

Chapter 4

Feasibility

Study

CHAPTER-4

FEASIBILITY STUDY

4.1 TECHNICAL FEASIBILITY

- Availability of high-performance LLM APIs (like Google Gemini Flash) for advanced natural language understanding and structuring.
- Python (FastAPI) and React ecosystems provide robust support for rapid full-stack prototyping and real-time data handling.
- Specialized libraries (like React Flow and Dagre) enable complex, automated graph visualization without building rendering engines from scratch.
- Modular backend architecture (like RapidAPI) allows for seamless integration of YouTube transcript extraction and AI processing modules.

4.2 ECONOMIC FEASIBILITY

- Uses industry-standard open-source frameworks such as React.js, FastAPI, and specialized visualization libraries.
- Leverages the Google Gemini API, which often provides a free tier for developers and students, minimizing API costs.
- No requirement for expensive local GPU hardware as the heavy AI computation is handled by cloud-based inference.
- Low long-term maintenance costs due to a modular design and reliance on established, supported libraries.

4.3 OPERATIONAL FEASIBILITY

- Intuitive frontend interface allows users to simply input a URL and receive a visual knowledge map.
- The system can be tested and iterated upon using any YouTube video with public captions without risk to external systems.
- Easy to update and maintain, as individual components like the AI prompt or layout engine can be tuned independently.

4.4 SOCIAL FEASIBILITY

- Significantly enhances student learning by providing a visual alternative to traditional, linear video consumption.
- Encourages collaborative learning by allowing students to save and share structured knowledge maps with peers.
- Builds industry-relevant skills in AI integration, graph theory, and modern full-stack development.
- Provides a practical tool for the academic community to improve retention and research efficiency.

4.5 ENVIRONMENTAL FEASIBILITY

- Entirely software-driven solution, generating no physical waste or specialized hardware disposal needs.
- Utilizes efficient API-based computing, which optimizes energy consumption compared to training local large-scale models.
- Operates on existing university or personal computer infrastructure, minimizing additional resource consumption.

4.6 SCALABILITY

- System architecture allows for the integration of different LLMs (e.g., Gemini Pro) to handle more complex or longer videos.
- Visualization dashboard can be expanded to include multi-video "Topic Clusters" or interactive quizzes based on the maps.
- The database structure supports scaling from individual user sessions to large-scale repositories of educational maps.
- Backend is designed to support cloud deployment (e.g., Docker/AWS) to accommodate a larger concurrent user base.

Chapter 5 Design And Implementation

CHAPTER-5

DESIGN AND IMPLEMENTATION

5.1 SYSTEM ARCHITECTURE

The system utilizes a decoupled, containerized hybrid-cloud architecture to ensure high availability and scalability across four major layers:

- **Frontend / Presentation Layer:** Built with React.js and Vite. It uses React Flow and Dagre for interactive node-edge graph rendering, alongside a dynamic Markdown panel for summaries.
- **Backend / Application Layer:** Developed using Python, FastAPI, and Docker. It manages asynchronous REST API routing, Pydantic data validation, and stateless JWT authentication.
- **AI / Processing Layer:** Uses the youtube-transcript-api to extract raw captions, piping them to the Google Gemini Flash LLM via engineered prompts for real-time semantic entity extraction.
- **Database / Storage Layer:** Powered by NeonDB (serverless PostgreSQL). It uses SQLAlchemy ORM and Alembic for secure schema migrations and user data storage.

5.2 WORKFLOW OF THE SYSTEM

The system's execution follows a highly automated pipeline:

1. **User Authentication and Input:** An authenticated user (via JWT) submits a target YouTube URL via the React dashboard.
2. **API Routing & Extraction:** The FastAPI backend verifies the request and programmatically fetches the raw subtitles using the youtube-transcript-api.
3. **Generative AI Processing:** The transcript is routed to the Gemini Flash LLM. The model performs semantic analysis to extract key concepts and logical relationships.
4. **Database Storage:** The structured graph data and associated metadata are securely saved to the NeonDB database via SQLAlchemy ORM for future retrieval.
5. **Dynamic Visualization:** The backend returns the JSON payload. The Dagre engine calculates spatial positioning, and React Flow renders the interactive dashboard.

5.3 DATA FLOW DIAGRAM & SYSTEM DESIGN

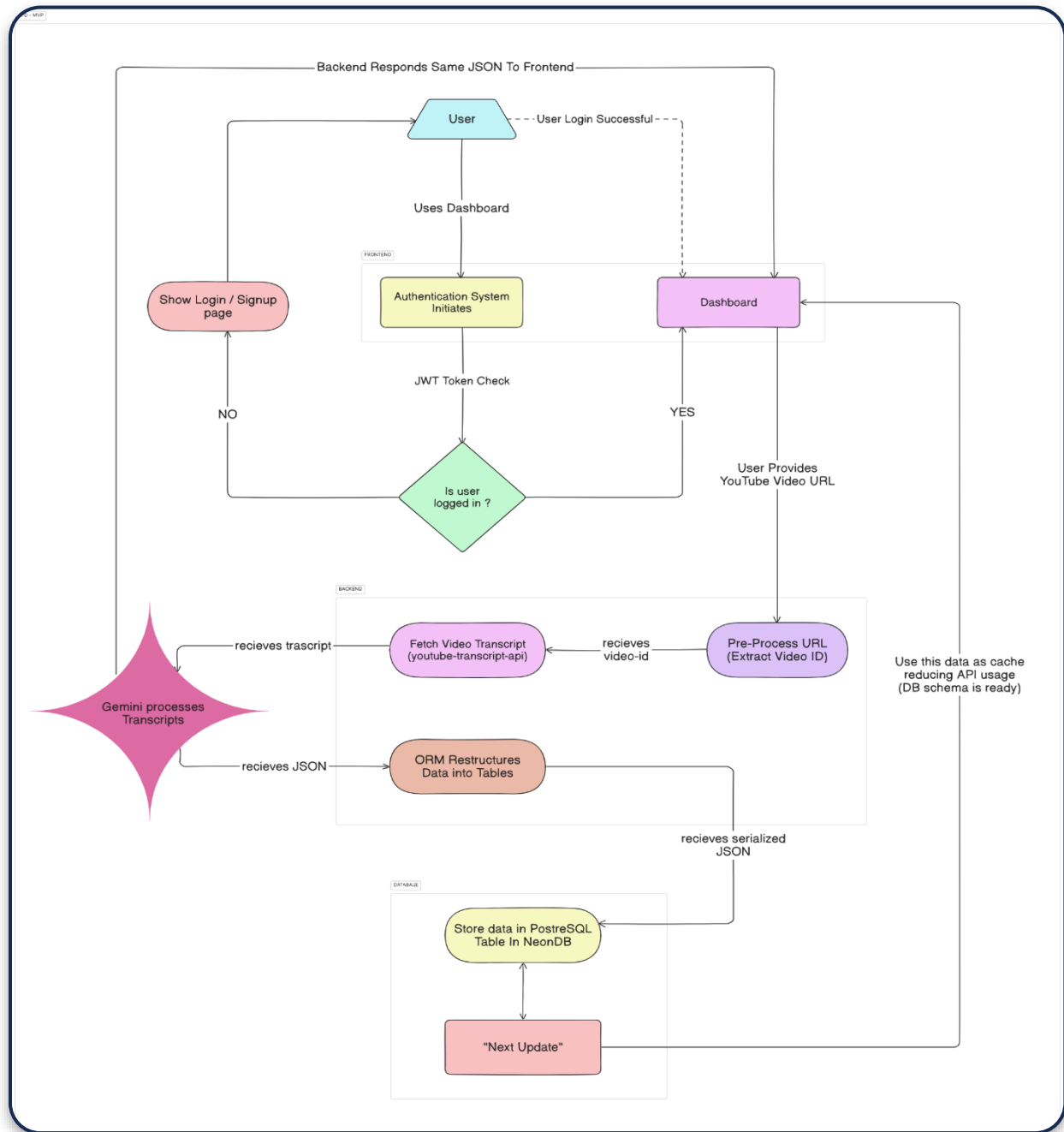


Fig. 5.1: Flowchart illustrating user validation and data flow in the TubeGraph

The DFD illustrates how data moves from a raw YouTube URL to structured visual insights. It highlights the interaction between URL validation, transcript extraction, the Gemini AI engine, the PostgreSQL backend, and the React-based TubeGraph interface..

5.4 GENERATIVE AI MODEL INTEGRATION

The core processing engine of the Video Knowledge Mapper relies on the Google Gemini Flash Large Language Model (LLM). Unlike traditional supervised classification models (like Random Forest) that output binary labels, Gemini is a multimodal generative model designed for advanced natural language understanding, semantic extraction, and text generation.

How LLM Inference Works in the System:

1. **Context Window:** The extracted video transcript is passed into the LLM's context window. Gemini Flash is specifically chosen for its large context capacity and low latency inference, allowing it to process thousands of words simultaneously.
2. **Semantic Parsing:** The model analyzes the transcript to identify recurring entities, key terms, and the overarching narrative structure.
3. **Relational Mapping:** It determines the logical relationships between the identified entities (e.g., determining that 'Photosynthesis' relates to 'Chloroplast').
4. **Structured Generation:** Based on the strict prompt instructions, the model formats the extracted data into a predefined JSON structure containing nodes and edges, alongside a Markdown summary.

Why Gemini Flash is Suitable for Transcript Analysis:

- **High Speed:** Optimized for rapid response times, essential for a real-time dashboard.
- **Instruction Following:** Highly capable of adhering to strict output constraints (like pure JSON formatting) without hallucinating extra conversational text.
- **Semantic Depth:** Can understand complex academic or technical jargon present in educational videos.

Parameters Used in Inference:

- **model:** "gemini-flash-latest" – Specifies the engine version.
- **response_mime_type:** "application/json" – Enforces the model to return valid, parsable JSON rather than raw text.

5.5 ENTITY RELATIONSHIP DIAGRAM & DATABASE DESIGN

TubeGraph utilizes NeonDB (serverless PostgreSQL) for cloud data persistence. The system employs SQLAlchemy ORM for secure, strictly-typed database interactions (preventing SQL injection) and Alembic for tracking schema migrations.

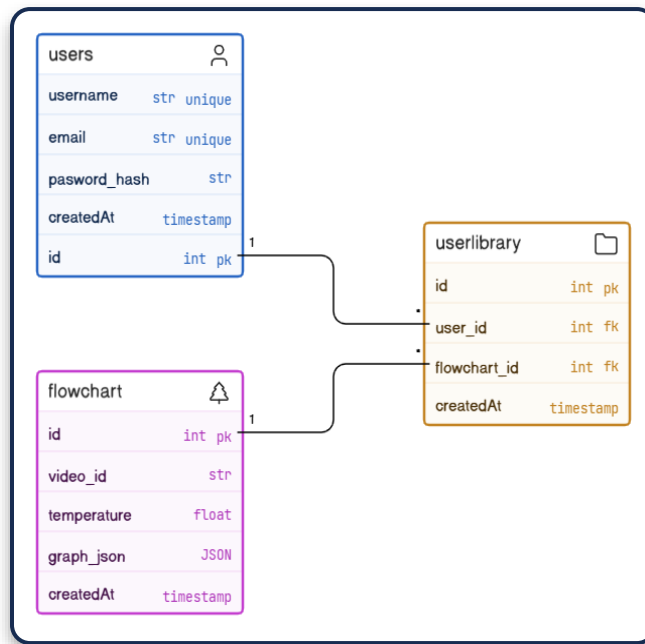


Fig. 5.2: Entity-Relationship (ER) Diagram of the TubeGraph Database Storage

The ER diagram illustrates the strict relational structure between user accounts and generated flowcharts, linked by the library junction table..

The database is structured around three core relational tables:

- **Users Table:** Manages authentication, securely storing username, email, hashed passwords, and timestamps.
- **Flowchart Table:** Stores the generated graph data as a native JSON column, alongside the video_id and LLM temperature.
- **UserLibrary Table:** A junction table enabling a many-to-many relationship using Foreign Keys (user_id, flowchart_id) with cascade-delete constraints to maintain automated data hygiene if an account is deleted.

5.6 BACKEND API IMPLEMENTATION

The backend is engineered using FastAPI (Python), leveraging asynchronous processing (async/await) to ensure heavy I/O-bound tasks.

Key Implementation Features:

- **Stateless Authentication:** User sessions are securely managed via JSON Web Tokens (JWT) sent in HTTP headers, eliminating the need for persistent server-side memory.
- **Strict Data Validation:** Employs Pydantic schemas to validate all incoming requests (URLs, temperatures) and AI-generated JSON payloads before any database interaction.
- **CORS Management:** Strictly configured middleware securely permits cross-origin HTTP traffic between the decoupled React frontend and the backend API.

5.7 FRONTEND & DASHBOARD DESIGN

The TubeGraph client-side interface is a React.js Single Page Application (SPA) compiled via Vite, featuring a responsive, dark-themed glassmorphic UI.

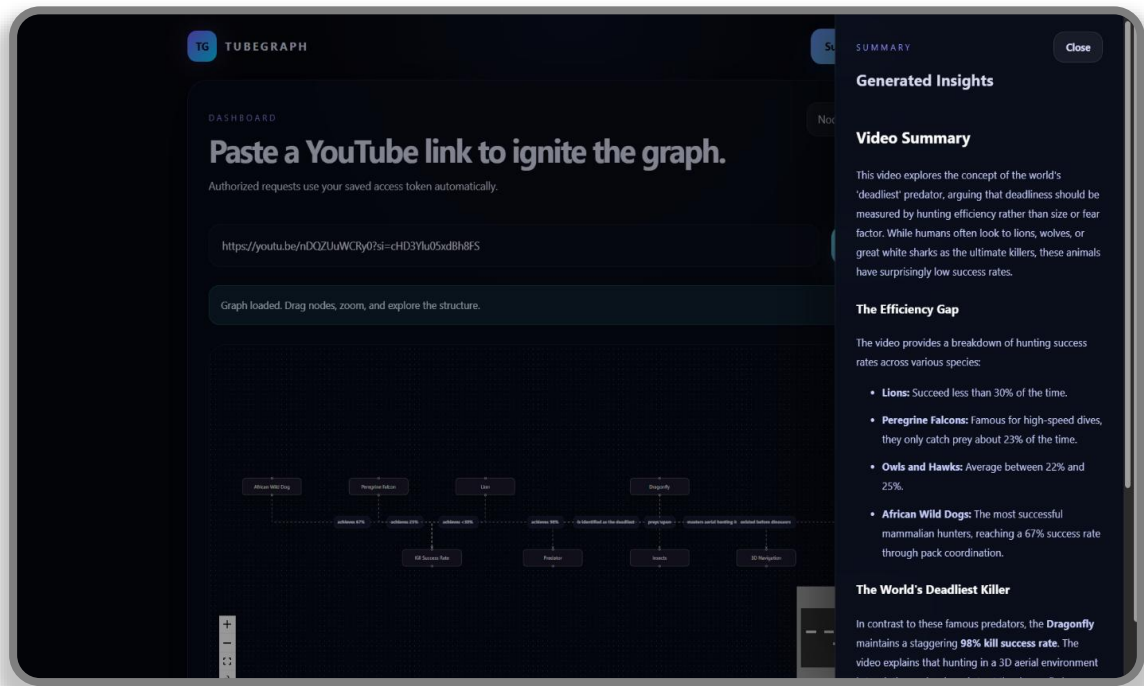


Fig. 5.3: Screenshot of the Interactive TubeGraph Dashboard

The dashboard features React Flow for interactive node-edge exploration and dynamically renders LLM-generated summaries.

Core Frontend Components:

- **Interactive Visualization:** Utilizes React Flow for node-based rendering and the Dagre engine for automatic hierarchical layout calculation, preventing node overlap.
- **Markdown Rendering:** Employs react-markdown to parse the AI-generated text into semantic HTML within a dedicated summary side-panel.
- **State Management & Routing:** Uses React Hooks (useState, useEffect) for async state management and validates JWTs in local storage to secure dashboard access.

5.8 TESTING AND VALIDATION

The system underwent a hybrid validation approach, ensuring software stability and predictable AI outputs:

- **API and Backend Testing:** Validated via FastAPI's Swagger UI (/docs) to ensure Pydantic schemas properly reject malformed URLs and JWT middleware blocks unauthorized requests.
- **Generative AI Prompt Validation:** The Gemini Flash model's temperature was optimized to balance strict JSON schema compliance with creative semantic extraction.
- **Frontend Integration Testing:** Tested React state resilience, ensuring accurate loading UI, secure token storage, and precise Dagre layout rendering without component overlap

5.9 FINAL SYSTEM OUTPUT

TubeGraph successfully delivers a robust, real-time pipeline for educational data extraction. The production-ready platform features:

- **Automated Knowledge Extraction:** Converts YouTube URLs into structured transcripts, utilizing Gemini Flash to extract core entities and semantic relationships..
- **AI-Driven Concept Mapping:** A React Flow dashboard for dragging, zooming, and exploring knowledge graphs alongside auto-generated Markdown summaries.
- **Secure Cloud Persistence:** A complete NeonDB and JWT-backed architecture allowing users to securely save and revisit generated learning materials.

Ultimately, TubeGraph successfully demonstrates a complete, production-ready, full-stack implementation of Generative AI in the educational technology domain.

Chapter 6

Conclusion

And

Future Work

CHAPTER-6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

- **Hands-on Learning with Generative AI and Visualization**
 - This project provided practical experience in integrating Large Language Models with interactive graph visualization. Building *TubeGraph* helped the team master transcript parsing, prompt engineering, and complex state management in React.
- **Successful Implementation of Automated Knowledge Mapping**
 - The application effectively transforms linear YouTube content into non-linear, hierarchical mind maps using the Gemini Flash model. It successfully visualizes concept flows in real-time, supporting enhanced retention and efficient information retrieval for learners.
- **Modular and Scalable Design**
 - The system was built using a layered architecture—FastAPI backend, Gemini processing engine, and React Flow frontend—making it easy to expand, debug, and upgrade. Each component, from the transcript extractor to the layout engine, can be independently optimized.
- **Educational Value and Skill Development**
 - Team members gained significant exposure to full-stack development, AI pipelines, and collaborative software engineering practices. The project bridged the gap between theoretical knowledge of AI and its practical application in EdTech tools.
- **Real-World Relevance**
 - The tool addresses a genuine need for structured learning in the era of video-based education. It serves as a foundation for future academic aids, automated study guides, and corporate knowledge management systems.

6.2 FUTURE WORK

- **Multi-Video Synthesis**
 - Future versions can support the ingestion of entire playlists to create comprehensive "Subject Maps" that link concepts across multiple videos.
- **Interactive Quiz Generation**
 - Adding a feature to automatically generate multiple-choice questions or flashcards based on the nodes of the mind map to test user retention.
- **Collaborative Mapping and Sharing**
 - Implementing a social layer where users can edit maps collectively or share their structured knowledge flows via unique URLs or social media.
- **Support for Multi-Modal Input**
 - Incorporating image and slide recognition from the video frames alongside transcripts to improve the context and accuracy of the generated nodes.
- **Advanced Layout and Customization**
 - Using libraries like D3.js or enhanced React Flow plugins to allow users to manually re-colour, group, or annotate nodes for a more personalized experience.
- **Cloud Synchronization and Persistence**
 - Full integration with cloud databases and user authentication (OAuth) to allow users to access their saved library of mind maps from any device.
- **Real-Time Live Stream Processing**
 - Enabling the tool to process live streams in real-time, allowing viewers to see a concept map grow dynamically as a lecture or webinar unfolds.
- **App Extension**
 - Developing a lightweight mobile version optimized for touch interactions, allowing students to review their knowledge maps on the go.

APPENDIX-1

GLOSSARY OF TERMS

(In alphabetical order)

A

API (*Application Programming Interface*) - A set of rules and protocols that allows the React frontend to communicate seamlessly with the FastAPI backend.

Authentication The process of verifying the identity of a user, handled in this system via email and password validation.

Authorization The process of determining whether an authenticated user has permission to access a specific resource or endpoint.

B

Backend The server-side part of an application that handles logic, data processing, and communication with databases or services.

Bearer Token A type of cryptographic token (like JWT) sent in the HTTP authorization header that grants the "bearer" access to protected backend routes.

C

Containerization (Docker) The packaging of software code along with its dependencies into a single, lightweight executable unit (a container) to ensure the application runs consistently across any computing environment.

CORS (*Cross Origin Resource Sharing*) - A security mechanism implemented by web browsers that controls how resources on a web server can be requested from another domain or port.

E

Environment Variables Configuration values stored outside the codebase, used to control application behavior across different systems or deployments.

Endpoint A specific URL where an API can be accessed by a client application to perform a distinct function (e.g., `/api/v1/graph/generate`).

F

FastAPI A modern, high-performance web framework for building APIs with Python, utilizing standard Python type hints for data validation.

Frontend The client-side, user-facing part of the application, built using React and React Flow to render interactive dashboards.

Full-Stack Development The practice of engineering both the frontend user interface and the backend server/database infrastructure of an application.

G

Git A version control system used to track code changes and support collaborative development.

GitHub A cloud-based platform used to host Git repositories and manage collaborative software projects.

GenAI (*Generative AI*) - Artificial intelligence systems capable of generating text, structure, or content from input data, such as the Gemini model analyzing transcripts.

H

HTTP (*Hyper-Text Transfer Protocol*) - The foundational protocol used by the World Wide Web, which governs the request-response communication between the React frontend and FastAPI.

I

Inference (AI) The phase in machine learning where a trained model (like Gemini Flash) makes predictions or generates text based on new, unseen input data (the transcript).

J

JSON (*Java-Script Object Notation*) - A lightweight data format used for storing and exchanging structured information, commonly used in APIs.

L

LLM (*Large Language Model*) - Advanced AI models trained on massive amounts of text data, designed to understand and generate human-like text responses (e.g., Google Gemini Flash).

M

Model (Database) A Python class defined using SQLAlchemy that maps directly to a relational database table structure (e.g., the **Flowchart** or **User** models).

Middleware Software that acts as a bridge between the operating system or database and applications. In FastAPI, middleware is used to handle CORS headers before a request reaches the core logic.

N

NeonDB A fully managed, serverless PostgreSQL cloud database platform used to store user accounts and historical graph data.

npm (*Node Package Manager*) - The default package manager for the JavaScript runtime environment, used to install and manage frontend dependencies like React Flow and Dagre.

O

ORM (*Object-Relational Mapping*) - A programming technique (like SQLAlchemy) that allows developers to interact with a database using Python code and object-oriented paradigms rather than writing raw SQL queries.

P

- PostgreSQL** An advanced, enterprise-class open-source relational database system. It serves as the underlying engine for the NeonDB cloud database.
- Payload** The actual data message transmitted within an API request or wrapped inside a JWT token, distinct from headers or metadata.

R

- ReactJS** A declarative, component-based JavaScript library used for building highly interactive user interfaces.
- React Flow** A specialized, highly customizable React library designed specifically for rendering interactive, drag-and-drop node-based diagrams and flowcharts.
- REST API** (*Representational State Transfer*) - An architectural style for designing networked applications, dictating how the FastAPI endpoints should be structured and accessed using standard HTTP methods (GET, POST).

T

- TubeGraph** The proprietary name of the interactive dashboard and software platform developed in this project, designed to visually map video knowledge.
- Transcript** The text-based, time-stamped record of all spoken words within a video, serving as the primary input data for the Gemini AI model.
- Token** A digital key used in web security. In this system, it specifically refers to a JSON Web Token (JWT) used to verify a user's logged-in state without needing to query the database on every request.

U

- UI** *User Interface* - The part of a system that users interact with directly, including dashboards, forms, and buttons.
- UX** *User Experience* - The overall experience of a person using the system or application application, particularly focusing on how intuitive, seamless, and pleasing it is to interact with.

REFERENCES

JOURNALS / RESEARCH PAPERS

1. Hao, Y., et al. (2023). "Large Language Models for Information Extraction: A Survey." *arXiv preprint*. (Discusses how LLMs like Gemini extract structured data and entities from unstructured text).
2. Davies, M. (2011). "Concept mapping, mind mapping and argument mapping: what are the differences and do they matter?" *Higher Education*, 62(3), 279-296. (A foundational academic paper on the effectiveness of node-edge visualizations in education).
3. Trajanoska, M., Stojanov, R., & Trajanov, D. (2023). "Enhancing Knowledge Graph Construction Using Large Language Models." *Institute of Electrical and Electronics Engineers (IEEE) Journals*.

BOOKS

1. Banks, Alex, and Eve Porcello. *Learning React: Modern Patterns for Developing React Apps*, O'Reilly Media, 2nd Edition, 2020.
2. Voron, Francois. *Building Data Science Applications with FastAPI*, Packt Publishing, 2021.
3. Tunstall, Lewis, Leandro von Werra, and Thomas Wolf. *Natural Language Processing with Transformers*, O'Reilly Media, 2022.

WEBSITES (with exact URL up to page)

1. FastAPI Official Documentation: <https://fastapi.tiangolo.com/>
2. React Flow Documentation: <https://reactflow.dev/docs/>
3. Google Gemini API Documentation: <https://ai.google.dev/docs>
4. YouTube Transcript API (Open Source Repository): <https://github.com/jdepoix/youtube-transcript-api>